Task 3
Longest Balanced Substring
Language: Python

Algorithm 1 — Brute Force (Non-Recursive)

```
FUNCTION longest_balanced(s)

    n ← length of s
    max_len ← 0

    FOR i from 0 to n - 1 DO

        freq ← empty dictionary

        FOR j from i to n - 1 DO

            c ← s[j]

            increase freq[c] by 1

            IF number of keys in freq > 2 THEN
                BREAK inner loop

            IF number of keys in freq = 2 THEN

                values ← all values in freq

                IF values[0] = values[1] THEN
                    max_len ← maximum(max_len, j - i + 1)

        END FOR

    END FOR

    RETURN max_len

END FUNCTION
```

```python
def longest_balanced(s):
    n = len(s)
    max_len = 0

    for i in range(n):
        freq = {}

        for j in range(i, n):
            c = s[j]
            freq[c] = freq.get(c, 0) + 1

            if len(freq) > 2:
                break

            if len(freq) == 2:
                values = list(freq.values())
                if values[0] == values[1]:
                    max_len = max(max_len, j - i + 1)

    return max_len

```

Time Complexity:

$O(n^2)$

The algorithm uses two nested loops to generate and check all possible substrings of the given string

1. Outer Loop

```
for i in range(n):
```

This loop selects the starting index of each substring. It runs n times, so its complexity is O(n).

2. Inner Loop

```
for j in range(i, n):
```

This loop expands the substring from index i to the end of the string

In the worst case, it runs:

- n times when i = 0
- n-1 times when i = 1
- n-2 times when i = 2
- ...

So total iterations are:

n + (n-1) + (n-2) + ... + 1 = n(n+1)/2

This simplifies to:

$O(n^2)$

Algorithm 2 — Recursive

```
FUNCTION longest_balanced_recursive(s, start, best)

    IF start >= length of s - 1 THEN
        RETURN best
    END IF

    freq ← empty dictionary

    FOR end from start to length of s - 1 DO

        c ← s[end]

        increase freq[c] by 1

        IF number of keys in freq > 2 THEN
            BREAK loop
        END IF

        IF number of keys in freq = 2 THEN

            vals ← values of freq

            IF vals[0] = vals[1] THEN
                best ← max(best, end - start + 1)
            END IF

        END IF

    END FOR

    RETURN longest_balanced_recursive(s, start + 1, best)

END FUNCTION
```

```python
def longest_balanced_recursive(s, start=0, best=0):

    if start >= len(s) - 1:
        return best

    freq = {}

    for end in range(start, len(s)):
        c = s[end]
        freq[c] = freq.get(c, 0) + 1

        if len(freq) > 2:
            break
        if len(freq) == 2:
            vals = list(freq.values())
            if vals[0] == vals[1]:
                best = max(best, end - start + 1)

    return longest_balanced_recursive(s, start + 1, best)

```

Time Complexity

The algorithm uses a recursive approach where for each starting index (start), it iterates through all possible ending indices (end) to form substrings.

- The outer recursion runs for O(n) starting positions.
- For each position, the inner loop may scan up to O(n) elements.
- Therefore, the total number of operations is:

$$n + (n-1) + (n-2) + \ldots + 1 = O(n^2)$$

Final Time Complexity:

$O(n^2)$

Comparison Between the Two Algorithms

Property	Algorithm1	Algorithm2
Type	Non-Recursive	Recursive
Outer iteration	for loop	function calls itself
Inner logic	same	same
Time Complexity	$O(n^2)$	$O(n^2)$
Space Complexity	O(1)	O(n) call stack
Base case needed	No	Yes
Stop condition	break on 3rd char	break on 3rd char
Memory usage	Lower	Higher
Code readability	Simple	Moderate

Time Complexity: Both algorithms have $O(n^2)$ time complexity. The outer loop (or recursive call) runs n

times and the inner loop runs up to n times per iteration, giving n x n = n^2 total operations.